Etude de l'influence de la température sur le coefficient de frottement statique d'un élastomère

PIROIRD Edgar

Session 2025/2026

N° SCEI 43646

Structure de l'étude

1. Problématique
2. Théorie
3. Influence de la température sur l'adhérence d'un pneu
4. Effet thermique des stries sur un pneu
5. Conclusion/Ouverture

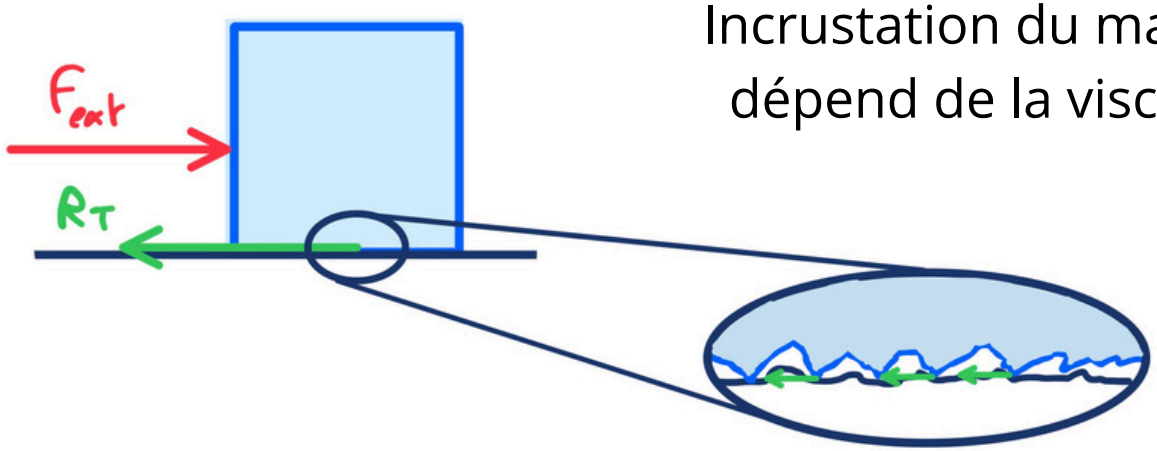
1 : Problématique

Comment la température influence-t-elle le coefficient de frottement statique d'un élastomère, et quels mécanismes physiques permettent d'expliquer cette variation ?



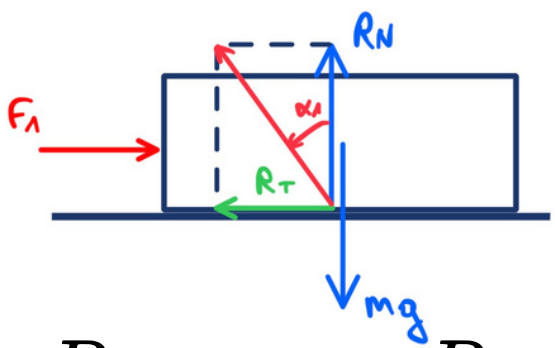
2 : Théorie

Indentation :



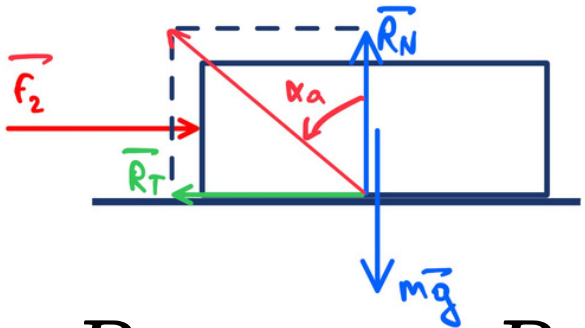
Incrustation du matériau dans son support, dépend de la viscoélasticité sur matériau

Coulomb :



$$R_t < \mu_s \times R_n$$

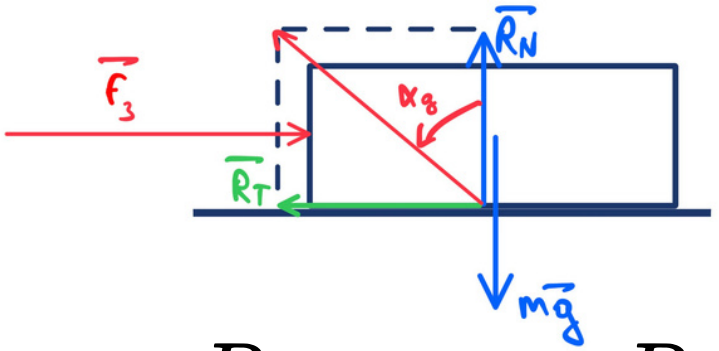
Coefficient d'adhérence



$$R_t = \mu_s \times R_n$$

Coefficient d'adhérence

$$\mu_s = tan\,(\alpha_a)$$



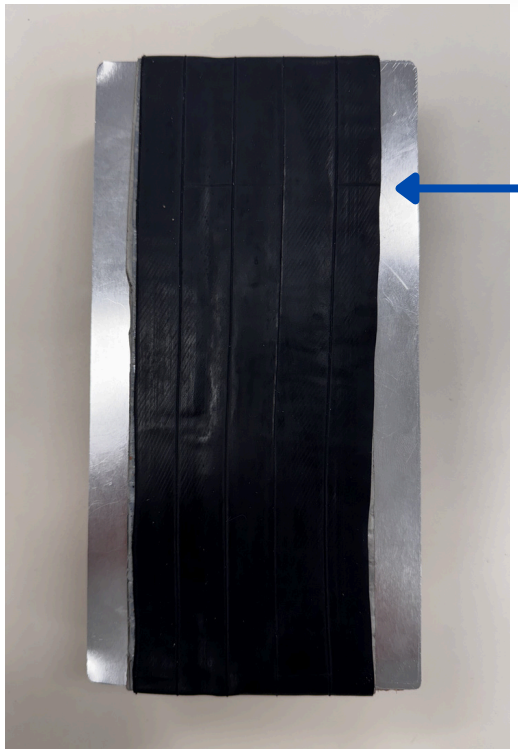
$$R_t = \mu_d \times R_n$$

Coefficient de frottement

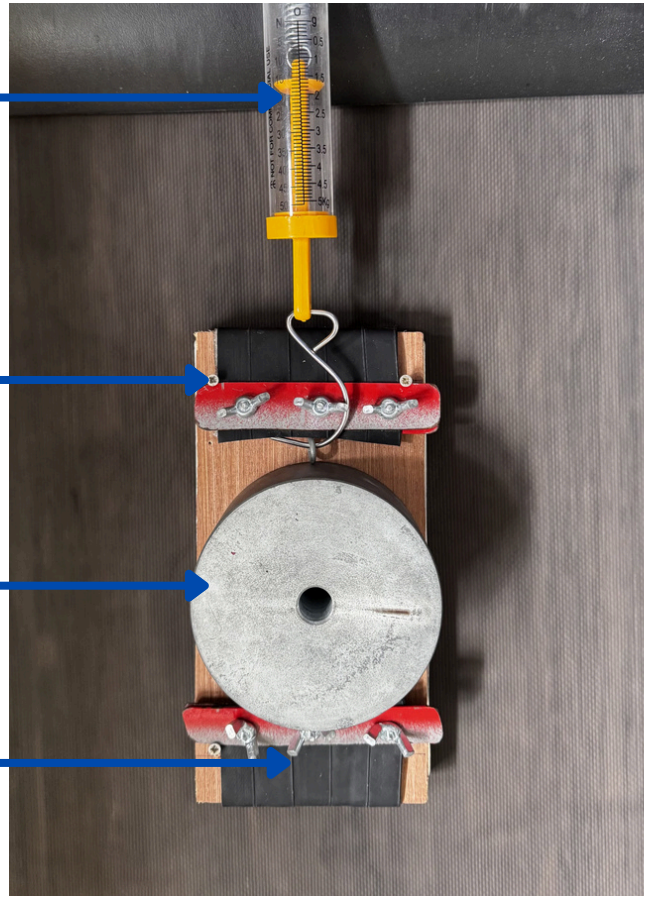
3 : Influence de la température sur l'adhérence d'un pneu : Système

Mesure de la température avec un pistolet thermique : ±1.5°C
Lecture valeur Rt au dynamomètre : ±0.5N

Masse totale du système : 2.34kg



Plaque de métal



Expérience : variation température système

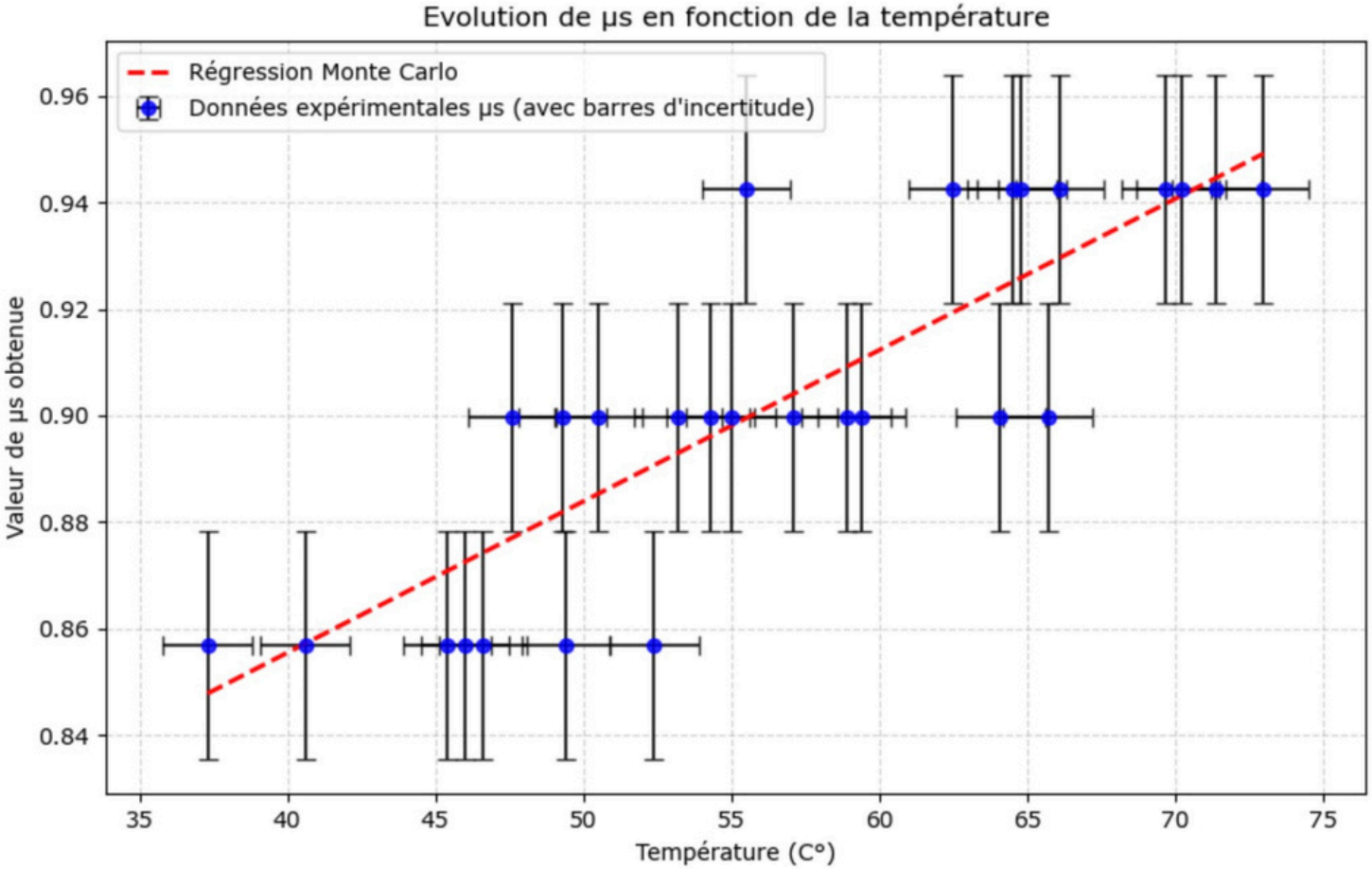
Système de chauffage : Etuve
Plage d'utilisation : 30-70°C





Serre-joint

Résultats



Pente :
y=0.00284*T+0.742

Incertitude (écart type) :
0.00025

Constat : µs est croissant avec la température.

Valeur expérimentale tabulée : µs entre 0.7 et 0.9

- Pour avoir le graphe : $\mu s = \frac{Rt}{Rn}$ ← Dynamomètre ← Poids (mg)

Expérience : variation surface de contact

Plaque :



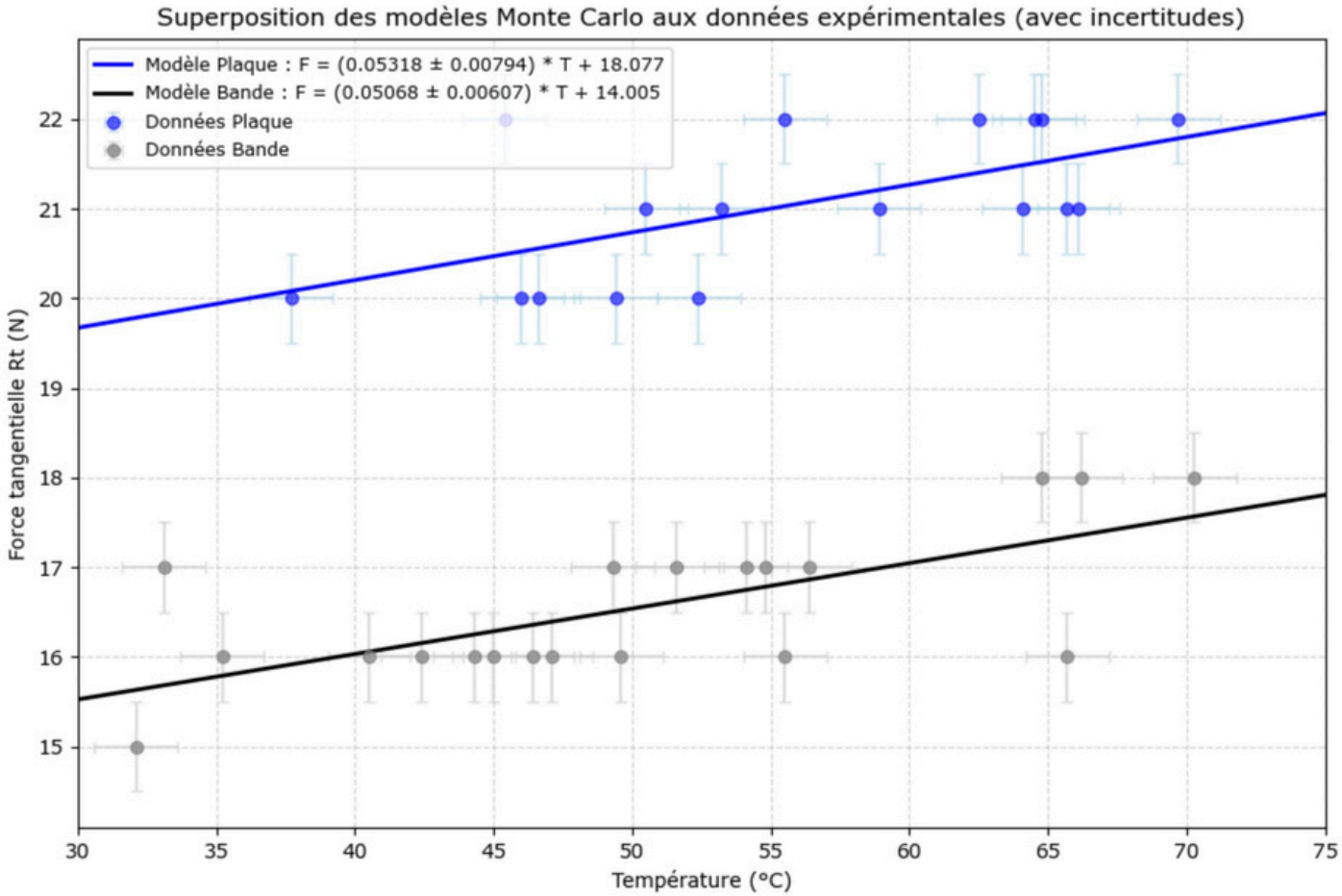
Bande :





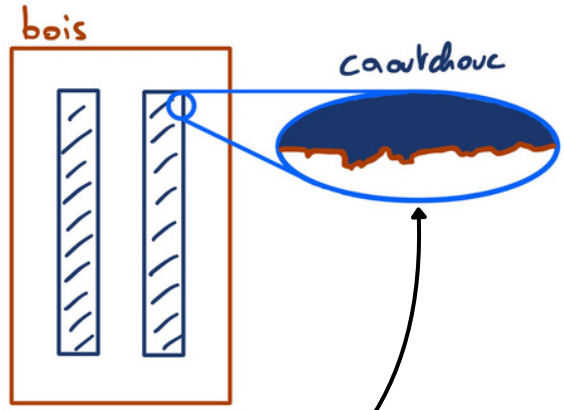
Masse globale toujours identique par ajout de masse supplémentaire (42g)

Résultats



Z_score sur les pentes : 0.29

Pentes compatibles



Les causes possibles des écarts d'ordonnées à l'origine :

- Biais surfacique : bois irrégulier - favorise la plaque (statistique)
- Possible saturation des microaspérites du système bande dû à la pression

Le coefficient d'adhérence est donc dépendant de la surface de contact

4 : Pneu "neige", preuve de concept

Expérience semi-quantitative pour mettre en évidence que les pneus neige montent plus rapidement en température, en partie grâce aux stries.







Type pneu	Température initiale	Température finale
Été	21.5	28.2
Hiver (strié)	21.8	29

Observations :

Température légèrement plus grande avec les stries

5 : Conclusion

Pour un élastomère

Température ↗ μs ↗ Vitesse ↗



Ouverture

Les différents paramètres important pour les pneus :

PNEU ÉTÉ



PNEU HIVER



PNEU 4 SAISONS



- Type de gomme (Transition vitreuse et indentation)
- Crantage en V (friction neige-neige et évacuation eau)
- Stries

Merci de votre attention !

Des questions ?

ANNEXES

Conditions expérimentales de l’expérience 2 :

Température ambiante : 21°C
Durée de l’expérience : 3min
Vitesse tapis : 8km/h

Type pneu	Température initiale	1 MIN	2MIN	Température finale
Été	21.5	23.4	26.6	28.3
Été	21.5	23.1	26.3	28.1
Été	21.7	23.6	26.5	28.4
Hiver	21.5	24.2	27	28.8
Hiver	22.1	24.4	27.3	29.2
Hiver	21.8	24.3	26.9	29

Incertitudes :

Incertitude pente : $$u(\mu s) = \mu s \times \sqrt{\left(\frac{u(Rt)}{Rt}\right)^2 + \left(\frac{u(Rn)}{Rn}^2\right)}$$

Z_score : $$\frac{|m1 - m2|}{\sqrt{(u(m1))^2 + (u(m2))^2}}$$

Avec m1 la pente moyenne S1 et m2 la pente moyenne S2

Les incertitudes-types u(m1) et u(m2) sur les pentes moyennes sont déterminées numériquement par la méthode de Monte-Carlo (calcul de l'échantillon d'écart-type sur N = 1000 simulations)

- Si Z < 2 : l'écart entre la plaque (S1) et la bande (S2) est statistiquement négligeable (les incertitudes de mesure suffisent à l'expliquer)
- Si Z > 2 : l'écart est significatif, ce qui prouve un effet physique

Conditions expérimentales pour système 1

Environnement :

• Expérience réalisée en salle fermée, à température ambiante contrôlée

• Absence de courants d'air et de rayonnement solaire direct

• Objectif : limiter les facteurs extérieurs autres que la température du caoutchouc

———

Support :

• Support plan rigide : Bois

• Surface horizontale vérifiée (niveau à bulle)

• Rugosité faible devant celle du caoutchouc afin de limiter l'influence des aspérités du support

Hypothèse : le frottement est dominé par le comportement du caoutchouc

———

Échantillon de caoutchouc :

• Matériau : caoutchouc synthétique

• Géométrie : pavé droit

• Surface de contact constante sur toute la série de mesures (19*7cm**2)

• Orientation conservée pour chaque essai

Variable étudiée : température, les autres paramètres sont maintenus constants

Charge normale :

• Masse ajoutée : m = 2,008kg

• Masse système : M = 371g

• Force normale :

N = mg ≈ (2,008+0,371)*9,81= 23,34N

Permet :

• d'assurer un contact stable et maximal (plat)

• d'améliorer la reproductibilité

• de réduire l'influence des vibrations

———

Contrôle de la température :

• Chauffage de l'échantillon par étuve

• Température imposée avant chaque mesure

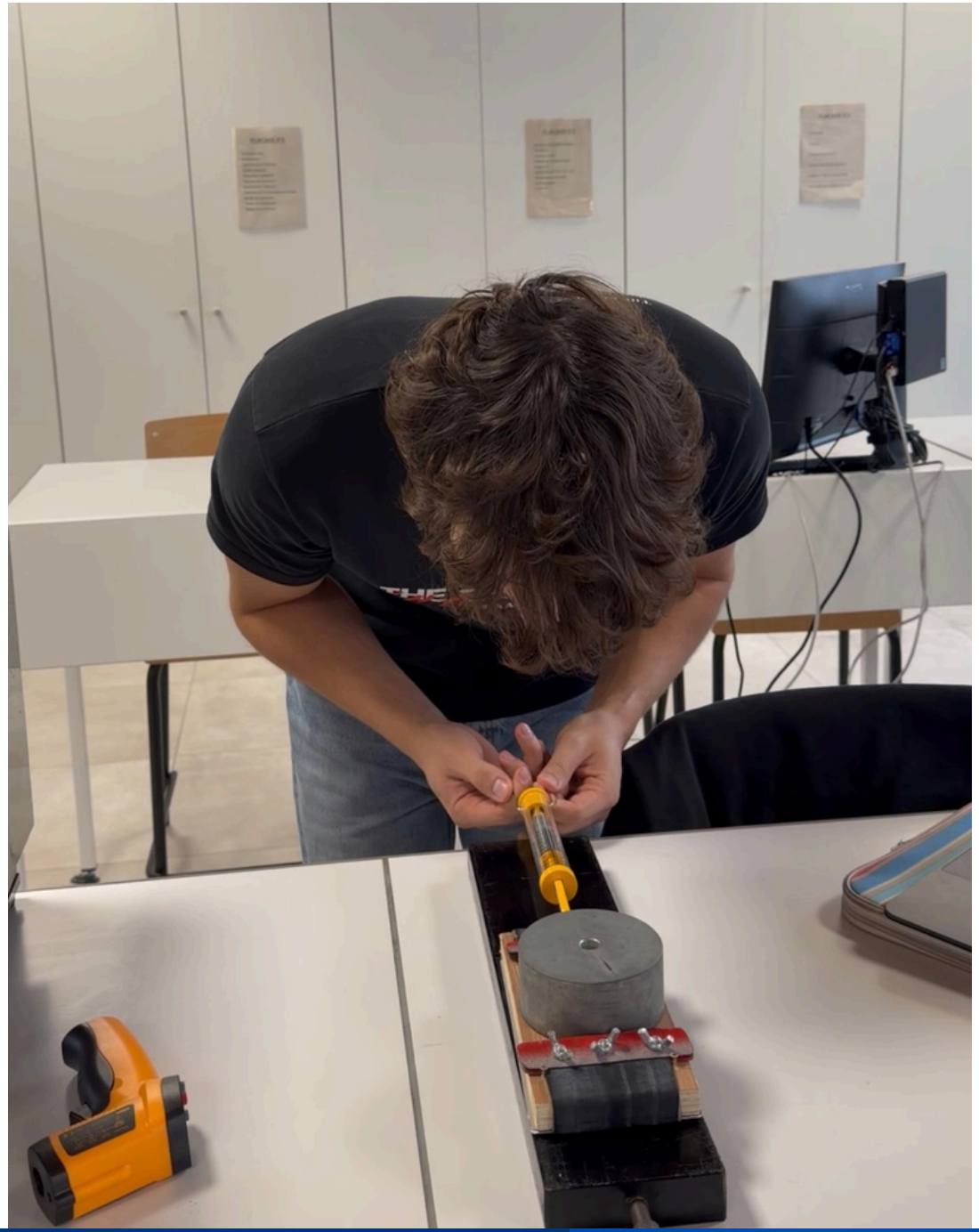
• Temps d'attente suffisant pour assurer une température homogène dans le caoutchouc

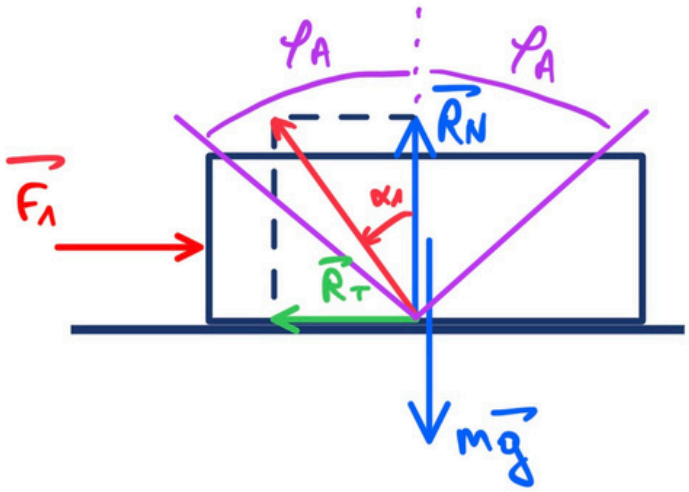
• Mesure de la température et pistolet infrarouge

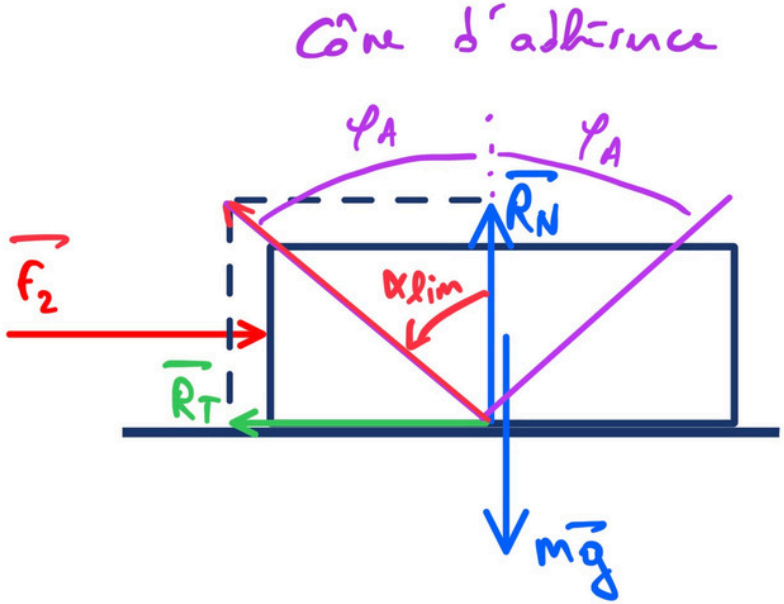
Hypothèse : la température du caoutchouc est uniforme au moment de la mesure

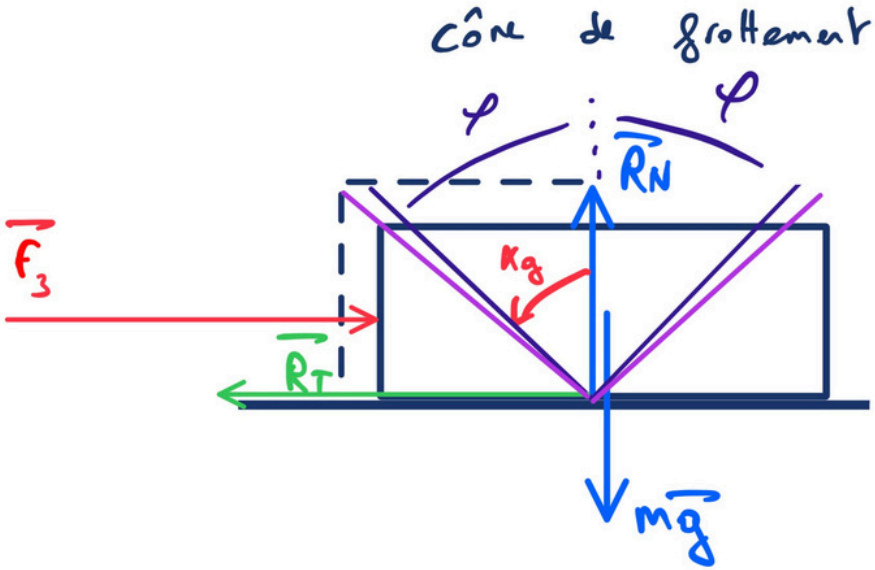
Pour l'expérience vérifiant que la surface de contact n'influence pas la valeur de la force tangentielle mesurée, on veillera bien à maintenir la masse globale du systeme constante. Pour cela on ajoute du poids supplémentaire. Qui pèse ici 42g





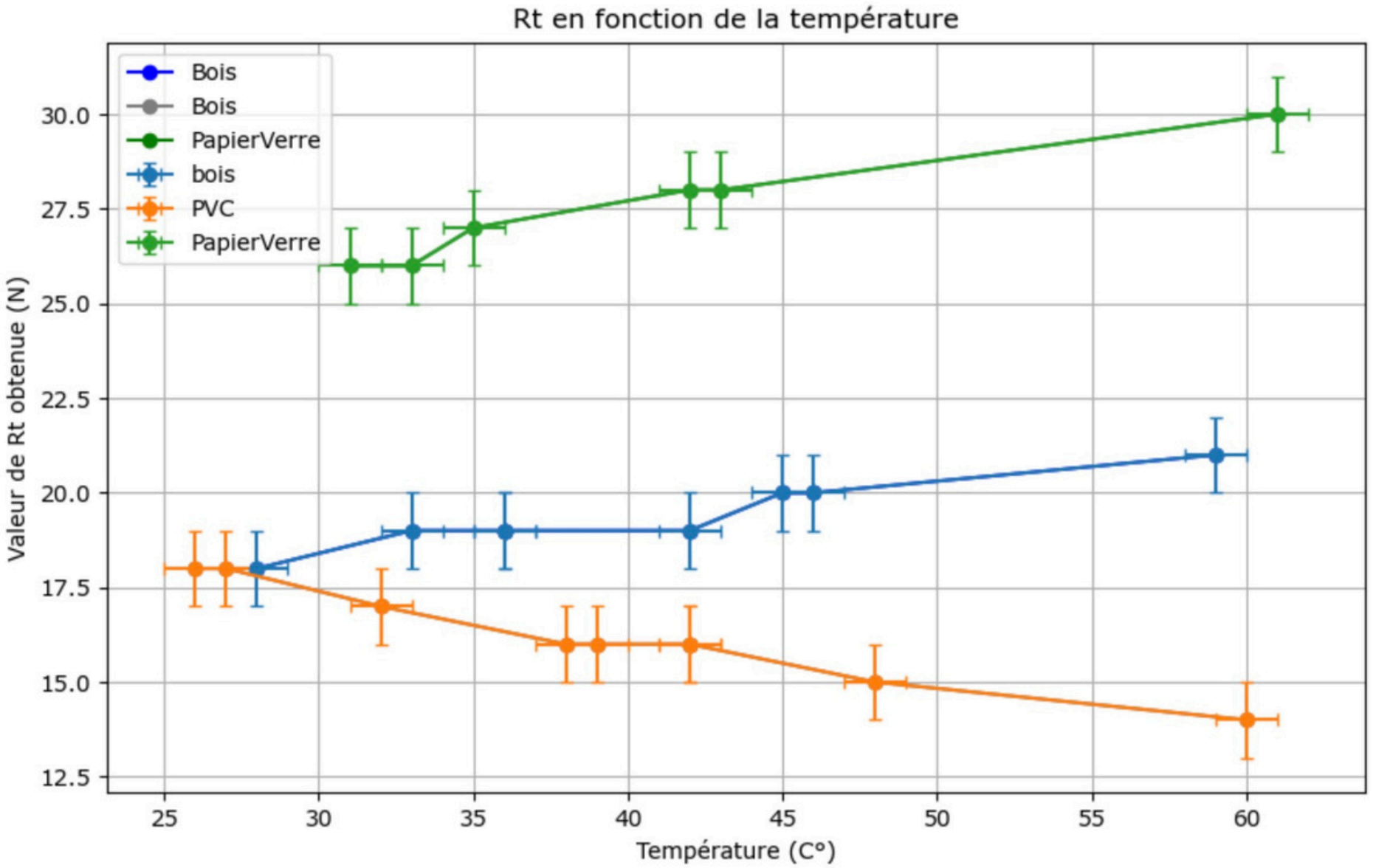

φ_A
φ_A
$\vec{R_N}$
$\vec{F_1}$
α_1
$\vec{R_T}$
$m\vec{g}$


Cône d'adhérence
φ_A
φ_A
$\vec{R_N}$
$\vec{F_2}$
α_{lim}
$\vec{R_T}$
$m\vec{g}$


Cône de frottement
φ
φ
$\vec{R_N}$
$\vec{F_3}$
α_g
$\vec{R_T}$
$m\vec{g}$


plaque de métal
masse additionnelle
vis
caoutchouc
plaque bois




Rt en fonction de la température
Bois
Bois
PapierVerre
bois
PVC
PapierVerre
Valeur de Rt obtenue (N)
30.0
27.5
25.0
22.5
20.0
17.5
15.0
12.5
25
30
35
40
45
50
55
60
Température (C°)

Monte Carlo 2 surfaces

```python
# 1. Données expérimentales
T1 = np.array([62.5, 64.1, 53.2, 65.7, 52.4, 69.7, 58.9, 64.8, 55.5, 50.5, 49.4, 46.6, 46, 45.4, 64.5, 66.1, 37.7])
F1 = np.array([22, 21, 21, 21, 20, 22, 21, 22, 22, 21, 20, 20, 20, 22, 22, 21, 20])

T2 = np.array([66.2, 70.3, 54.1, 47.1, 49.3, 46.4, 42.4, 35.2, 32.1, 64.8, 55.5, 54.8, 51.6, 49.6, 44.3, 45, 40.5, 33.1, 65.7, 56.4])
F2 = np.array([18, 18, 17, 16, 17, 16, 16, 16, 15, 18, 16, 17, 17, 16, 16, 16, 16, 17, 16, 17])

# 2. Paramètres de la simulation de Monte Carlo
N = 10000          # Nombre de simulations
delta_T = 1.5      # Demi-largeur de l'incertitude sur T
delta_F = 0.5      # Demi-largeur de l'incertitude sur F

# --- Fonction Monte Carlo modifiée ---
def simulation_monte_carlo(T_data, F_data, N_sim, dT, dF, nom_systeme="S"):
    pentes_simulees = np.zeros(N_sim)
    ordonnees_simulees = np.zeros(N_sim)

    for i in range(N_sim):
        # On génère une perturbation aléatoire uniforme pour chaque point
        T_perturbe = T_data + np.random.uniform(-dT, dT, size=len(T_data))
        F_perturbe = F_data + np.random.uniform(-dF, dF, size=len(F_data))

        # Ajustement linéaire (polyfit de degré 1)
        pente, ordonnee = np.polyfit(T_perturbe, F_perturbe, 1)
        pentes_simulees[i] = pente
        ordonnees_simulees[i] = ordonnee

    # Calculs statistiques des moyennes et incertitudes
    pente_moyenne = np.mean(pentes_simulees)
    incertitude_pente = np.std(pentes_simulees, ddof=1)
    ordonnee_moyenne = np.mean(ordonnees_simulees)

    # Formatage propre de l'équation de la droite (F = (m ± u_m)*T + b)
    signe = "+" if ordonnee_moyenne >= 0 else "-"
    equation_str = f"F = ({pente_moyenne:.5f} ± {incertitude_pente:.5f}) * T {signe} {abs(ordonnee_moyenne):.3f}"

    return pentes_simulees, pente_moyenne, incertitude_pente, ordonnee_moyenne, equation_str

# 3. Exécution pour les deux systèmes
pentes_S1, m1, u_m1, b1, eq_S1 = simulation_monte_carlo(T1, F1, N, delta_T, delta_F, "S1")
pentes_S2, m2, u_m2, b2, eq_S2 = simulation_monte_carlo(T2, F2, N, delta_T, delta_F, "S2")

# 4. Calcul du Z-score entre les deux pentes
z_score = abs(m1 - m2) / np.sqrt(u_m1**2 + u_m2**2)
```

Affichage 2 surfaces

```python
# Génération des points pour les lignes de modèle
T_modele = np.linspace(30, 75, 100)
F1_modele = m1 * T_modele + b1
F2_modele = m2 * T_modele + b2

plt.figure(figsize=(11, 7))

# 1. Points S1 : Nuage de points avec barres d'incertitude sur chaque point
plt.errorbar(T1, F1, xerr=delta_T, yerr=delta_F, fmt='o', color="blue", ecolor="lightblue", capsize=3, alpha=0.6, label="Données Plaque")

# 2. Points S2 : Nuage de points avec barres d'incertitude sur chaque point
plt.errorbar(T2, F2, xerr=delta_T, yerr=delta_F, fmt='o', color="grey", ecolor="lightgrey", capsize=3, alpha=0.7, label="Données Bande")

# 3. Droites de régression moyenne intégrant les équations calculées dans la légende
plt.plot(T_modele, F1_modele, color="blue", linestyle="-", linewidth=2,label=f"Modèle Plaque : {eq_S1}")
plt.plot(T_modele, F2_modele, color="black", linestyle="-", linewidth=2,label=f"Modèle Bande : {eq_S2}")

# Habillage et mise en page
plt.title("Superposition des modèles Monte Carlo aux données expérimentales (avec incertitudes)", fontsize=12)
plt.xlabel("Température (°C)")
plt.ylabel("Force tangentielle Rt (N)")
plt.xlim(30, 75)
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend(loc="upper left", fontsize=9)

plt.show()
```

On doit tracer une ligne de modèle car nos mesures ne sont pas directement comparables car pas aux mêmes températures.

Monte Carlo Courbe seule

```python
import matplotlib.pyplot as plt
import numpy as np

# --- Données : petit bois ---
T1=[62.5,64.1,53.2,65.7,52.4,69.7,58.9,64.8,55.5,50.5,49.4,46.6,46,45.4,64.5,66.1,54.3,70.2,71.4,73,59.4,57.1,55,49.3,47.6,40.6,37.3]
F1=[22,21,21,21,20,22,21,22,22,21,20,20,20,20,22,22,21,22,22,22,21,21,21,21,21,20,20]

T1 = np.array(T1)
F1 = np.array(F1)
Rn = 23.34

# Calcul de μs1 (coefficient de frottement statique)
mus1 = F1 / Rn

# Tri des données pour le graphique
indices1 = np.argsort(T1)
T1_trie = T1[indices1]
mus1_trie = mus1[indices1]


# Fonction Monte Carlo
def simulation_monte_carlo(T_data, Y_data, N_sim, dT, dY, nom_pente="S"):
    pentes_simulees = np.zeros(N_sim)
    ordonnees_simulees = np.zeros(N_sim)

    for i in range(N_sim):
        # Perturbation aléatoire uniforme pour chaque point
        T_perturbe = T_data + np.random.uniform(-dT, dT, size=len(T_data))
        Y_perturbe = Y_data + np.random.uniform(-dY, dY, size=len(Y_data))

        # Ajustement linéaire (degré 1)
        pente, ordonnee = np.polyfit(T_perturbe, Y_perturbe, 1)
        pentes_simulees[i] = pente
        ordonnees_simulees[i] = ordonnee

    # Calcul des moyennes et incertitudes-types
    pente_moyenne = np.mean(pentes_simulees)
    incertitude_pente = np.std(pentes_simulees, ddof=1)
    ordonnee_moyenne = np.mean(ordonnees_simulees)
```

Affichage variation T

```
# --- Graphique ---
plt.figure(figsize=(10, 6))

# 1. Tracé des données expérimentales (points non reliés avec incertitudes)
plt.errorbar(T1_trie, mus1_trie, xerr=incertitude_T, yerr=incertitude_mus, fmt='o', color="blue", ecolor="black", capsize=4, alpha=0.8,label="

# 2. Tracé de la droite de régression Monte Carlo
T_droite = np.linspace(min(T1), max(T1), 100)
mus_droite = pente_m1 * T_droite + ordonnee_m1

# On intègre dynamiquement l'équation complète dans la légende du graphique !
plt.plot(T_droite, mus_droite, color="red", linestyle="--", linewidth=2, label=f"Régression Monte Carlo ({eq_S1})")

plt.title("Evolution de µs en fonction de la température")
plt.xlabel("Température (C°)")
plt.ylabel("Valeur de µs obtenue")
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(loc="upper left") # "upper left" évite que la longue légende ne cache tes points
plt.show()
```